

Brain-to-Text’25: Decoding Speech from Neural Activity with Context-Aware Acoustic Representations and Large Language Models

Rishabh Sood, Ozgur Mert Emir, Qianyi Miao, Rahul Kumar Agrawal
Georgia Institute of Technology
Department of Computer Science

rsood39@gatech.edu, oemir3@gatech.edu, qmiao32@gatech.edu, ragrawal306@gatech.edu

Abstract

*This work addresses the critical challenge of restoring communication for individuals with neurological conditions like ALS through brain-computer interfaces (BCIs). While state-of-the-art approaches have demonstrated high accuracy 2.5% WER (Word Error Rate), residual challenges remain concerning neural variability and optimizing the modular decoding architecture. We propose several deep learning architectures to further analyze underlying data and models, building upon a baseline that employs a 5-layer GRU model trained with CTC loss to map neural features to phoneme posteriors, followed by WFST n-gram decoding. Alongside, we explore enhancing the system through Lattice Rescoring with Large Language Models (LLMs), as well as including fine-tuned GPT-2 and T5-Base architectures for direct phoneme-to-text generation. We also incorporate strategies like data augmentation, ablation studies, re-ranking methods to reduce the WER on the validation set as compared to the standalone performance of these models. The project is inspired from a Kaggle challenge: **Brain-to-Text ’25***

1. Introduction/Background/Motivation

Neurological conditions such as ALS and brainstem strokes can severely affect the human ability to move and speak, significantly increasing their risk of isolation, depression and decreased quality of life. For these individuals, transforming thoughts of speech directly into external output, such as text on a screen, is critical for restoring their connection to the world. Our primary objective is to advance existing technology, specifically brain-computer interfaces (BCIs) to achieve accurate and faster conversion of intended speech, derived from neural activity, into fluent and readable sentences. Although prior efforts demonstrated 97.5% decoding accuracy [2], our project aims to further reduce word error rates (WER) using deep learning

approaches.

Communication for people with severe dysarthria (speech impairment) has historically relied on augmentative and assistive communication (AAC) technologies, such as eye trackers or head trackers [6]. However, these methods are severely limited because they have low information-transfer rates and become increasingly difficult for patients to use as they lose voluntary muscle control. Brain-computer interfaces offer a promising alternative by directly decoding the user’s intended speech from neural signals. Early attempts to develop brain-to-text systems faced limitations, requiring extensive training requirements and yielding limited accuracy. Two previous major studies reported median word error rates of 25.5% (with a 1024-word vocabulary) [16] and 23.8% (with a 125,000-word vocabulary) [22], requiring roughly 17 hours of recorded data for training. The current state-of-the-art approach, which serves as the immediate baseline for this project, significantly overcame these limitations. This used advanced surgical implantation gathering high resolution neural data as well as deep learning architecture for sequential decoding of neural signals of phonemes (smallest, distinct units of sound) followed by n-gram language models for converting predicted phonemes into coherent sentences [2]. This intracortical speech neuroprosthesis achieved 99.6% accuracy on a 50-word vocabulary on the first day of use (requiring just 30 minutes of calibration). After just 1.4 additional hours of system training, the system achieved 90.2% accuracy using a 125,000-word vocabulary. Eventually, this system sustained 97.5% accuracy over 8.4 months. While the performance is better than automated speech recognition for able-bodied speakers (approximately 5% WER), challenges remain, including day-to-day neural variability and optimizing the combined modular architectures of neural activity to phonemes and phonemes to text. This project relies on the Card et al. (2025) dataset provided by the UC Davis Neuroprosthetics Lab [2, 1]. The core instances represent cortical neural activity recorded from a 45-year-old man with ALS who has tetraparesis and severe dysarthria. Data was

collected following the surgical implantation of four micro-electrode arrays into his left ventral precentral gyrus, an area of cortex responsible for speech. These arrays recorded neural activity from 256 electrodes at high resolutions of 20ms, as he attempted to speak in both prompted and unstructured conversational contexts. The neural activity was transmitted to external computers. The target labels are the intended speech, which is ultimately output as text on a screen. The data was collected as part of BrainGate2 clinical studies with informed consent from the participant. A strong limitation of the dataset is that the entire data comes from recording of 1 individual, missing the human oriented variability of neural signals. A detailed breakdown of the competition dataset, shared by the problem authors can be viewed [here](#).

To train the downstream component converting predicted phonemes into final sentences, we aggregated additional publicly available corpora to create a diverse training set. This comprises **OpenWebText2**, **Switchboard** and **Harvard Sentences** [7, 9, 14], helping generate temporal diversity and high-quality coverage of English phoneme distributions.

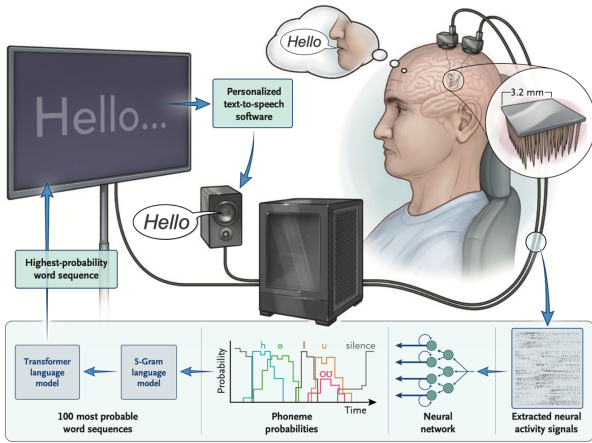


Figure 1: Overview of the Brain-to-Text baseline decoding pipeline which achieves a 6.70% word error rate on the held-out test set. *Image Source Credit:* [2]

If successful, this technology provides a means to restore conversational communication. The current baseline enables a communication rate of 32 words per minute [2]. By making the BCI output more fluent and accurate, the future participants could engage in personal conversations, interact with health-care professionals and colleagues, and perform tasks necessary for their occupation, such as participating in video conferencing meetings, writing documents and email messages, upgrading their quality of life.

2. Approach

2.1. Baseline: CTC + WFST n -gram Decoding

As a reference system, we follow the CTC-WFST decoding pipeline used in the original brain-to-text neuroprosthesis by Card et al. and in the official Brain-to-Text '25 baseline implementation.[2, 3] Neural features from the participant's speech motor cortex are first mapped to frame-level phoneme posteriors by a 5-layer GRU model trained with CTC loss, using day-specific linear input layers and overlapping temporal patches as described in the public repository and supplemental materials.[3]

We use a Kaldi-style WFST decoder over a TLG graph [8, 10]. The decoder performs beam search to extract the N-best list, which is then rescored by a sentence-level LLM (see Appendix A.1 for formal definitions)

In the following subsections, we discuss our approaches to the two stages (neural-encoding $\rightarrow$ phoneme $\rightarrow$ text), while some build upon the discussed pipeline, others explore industry best practices / aim to perform comparison with other architectures such as attention-based decoding.

2.2. Stage 1: Neural Encoding to Phonemes

We begin our pipeline by transforming 512-dimensional neural feature sequences from the original dataset into phoneme sequences that will later be used for word prediction. Because the neural recordings are inherently temporal, we treat this as a Seq2Seq mapping problem and explore six temporal model architectures including unidirectional and bidirectional vanilla RNN, GRU, and LSTM.

To improve robustness to noisy and nonstationary neural signals, we incorporate a set of data augmentation strategies during training. In particular, we apply Gaussian smoothing to reduce high-frequency noise, random time mask to simulate transient dropouts, random channel dropout to mimic the degradation of individual electrodes, and small random time shifts to model timing jitter in the neural recordings.

2.3. Stage 2: Phoneme to Text

To convert decoded phonemes from the previous stage into natural language text, we explore multiple phoneme-to-text decoding strategies. Throughout this section, the term decoding refers to the process of mapping a sequence of phonemes to a textual sentence, and should not be confused with architectural decoder components in neural sequence models.

As a baseline, we build upon the approach provided by the problem authors, which combines a pre-trained n -gram language model with lattice beam search and large language model (LLM)-based sentence probability scoring. We extend this baseline by incorporating lattice re-scoring, with detailed analysis provided in the Experiments and Results section.

2.3.1 Baseline + Lattice Rescoring with LLMs

To extend the CTC+WFST system described in §2.1, we follow the approach used in recent speech-BCI work and in the competition baseline implementation.[2, 3] Rather than modifying the acoustic model or rebuilding the WFST graph, we integrate a pretrained transformer language model (LLM) directly into the decoding stage by rescoring the *full WFST lattice*. This allows us to bring in strong long-range linguistic structure—which n -gram LMs cannot capture—while preserving the acoustic and lexical constraints already encoded in the lattice.

The intuition is straightforward: the CTC+WFST decoder already produces a rich set of acoustically plausible candidate paths. Many of these paths differ only in grammatical or semantic plausibility. Instead of restricting the LLM to a small N -best list (as in classic LLM rescoring), we treat the entire lattice as a constrained word graph and run a beam search in which each hypothesis accumulates (i) CTC acoustic cost, (ii) n -gram LM/lexicon graph cost, and (iii) an LLM prefix probability. The LLM thereby guides search toward globally coherent sentence structures without overriding the acoustic evidence.

To enable this, we extend the baseline C++ decoder so that it exports the full Kaldi lattice (states, arcs, word labels, and WFST weights) to Python, exactly as produced during decoding.[8, 3] The resulting Python-side search algorithm interprets the lattice as a directed acyclic word graph and performs a log-linear beam search combining acoustic, graph, and LLM scores. The full formulation—lattice representation, hypothesis structure, scoring function, and beam-search algorithm—is given in Appendix A.2.

Empirically, integrating the LLM at the lattice level produces substantially more grammatical and contextually appropriate transcriptions than both: (i) the baseline CTC+WFST system, and (ii) the baseline sentence-level LLM reranking of a fixed N -best list. This confirms that stronger long-range language modeling is beneficial, and that allowing the LLM to explore the entire lattice (rather than a restricted subset) is critical for improved decoding quality.

2.3.2 Beyond the Baseline: Attention based decoding

We now investigate two additional approaches that frame phoneme-to-text conversion as a sequence-to-sequence learning problem. Specifically, we fine-tune a decoder-only architecture (GPT-2 [18]) and an encoder-decoder architecture (T5-Base [19]) for direct phoneme-to-text generation. This sub-section focuses on the methodology and data preparation for these sequence-to-sequence models.

Dataset Preparation

To construct a diverse and temporally broad corpus for phoneme-to-text modeling, we aggregate data from three complementary sources alongside the dataset shared by the authors.

First, we sample text data from **OpenWebText2** [7, 20], selecting up to 500 samples per month spanning the years 2005–2020, resulting in a total of 88,499 documents. The sampled texts are segmented into sentences, with sentences containing URLs or residual HTML markup filtered out. This procedure ensures both temporal diversity and textual cleanliness while limiting topical bias.

Second, we incorporate the **Harvard Sentences dataset** [14], a curated collection of 720 phonetically balanced utterances. Although small in size, this dataset provides high-quality coverage of English phoneme distributions and serves as an important anchor for phonetic diversity.

Third, we include a cleaned version of the **Switchboard conversational speech dataset** [9, 17], contributing approximately 0.1 million samples. All utterances containing `< LAUGH >` annotations are removed. We preserve the original train/validation/test split provided with the dataset to maintain consistency with prior work.

Finally, All Phoneme - Sentence pairs from the **Brain-To-Text-25** [2, 1] trainset were also included in this dataset. These samples were directly added without filtering.

Training Data Structure

For **GPT-2**, phonemes are treated as atomic tokens (i.e. all phonemes were added as special tokens to the tokenizer / the model’s embedding layer). The phoneme input is capped at 800 characters, while the textual input/output is limited to 200–250 **GPT-2 tokens** (approximately 800–1000 characters). Samples exceeding the context window are truncated. We introduce special tokens `<bos>`, `<sep>`, and `<eos>` to delimit phoneme and text segments.

All sequences are padded to the maximum context length, with attention masks set to zero for padded positions. For **T5**, we maintain the same filtering and truncation strategy to ensure fair comparison. *No additional special tokens are introduced.* Inputs are formatted as *natural-language prompts* and the target text is provided directly as the model label.

For more details on training data format for both models, please refer to (Appendix A.5.3).

Train-Val-Test Split

The final dataset, after preprocessing and filtering, consisted of 1,223,469 training samples, 24,601 validation samples, and 32,010 test samples for phoneme to text modeling.

Data Augmentation

To improve robustness against phoneme recognition and alignment errors, we design a noise injection strategy guided by empirical error analysis. Specifically, we analyze predictions from our RNN architecture-benchmark (bidirectional LSTM) model on the validation set (more details on this in the *Experiments & Results* section), which yields an overall error rate of 9.9%. The dominant error modes include voiced-voiceless confusions (e.g., $D \leftrightarrow T$, $Z \leftrightarrow S$), vowel confusions (e.g., $IH \leftrightarrow AH$, $IY \leftrightarrow IH$), and word-boundary (" | ") deletions. Frequent phoneme insertions (notably N, AH, T, D, and S) are also observed.

Mirroring this distribution, we apply targeted augmentation to create noisy training examples. For each augmented sample, approximately 5% of phonemes are **substituted** based on the observed confusion matrix (Appendix: Fig. 8), 3% are **deleted** (favoring commonly dropped tokens), and 2% are **inserted** using frequently over-predicted phonemes (Distribution on Validation Set: (Appendix: Fig 9)). This strategy injects realistic error patterns and improves model tolerance to phonetic ambiguity. A detailed analysis script with the final confusion matrix can be found in our code repository [21]. To illustrate the effect of predicting voiced entities, we further attach an illustration on voiced-unvoiced confusion patterns (Appendix: Fig 10).

2.3.3 Acoustic Reranking

Upon fine-tuning our sequence-to-sequence model candidates on the above dataset, we then apply acoustic reranking to the model outputs during validation / test phase. This involves:

1. **Extracting N-best phoneme hypotheses** via CTC beam search on predicted logits from the neural encoding to phoneme layer, with their confidence probabilities (acoustic score).
2. For each phoneme hypothesis, we then **Generate Predictions** using our fine-tuned sequence-to-sequence models.
3. Next, we **Score each generated prediction** with a pre-trained language model (distillgpt2 / mistral-7b)
4. Finally, we **Combine acoustic (α) + LLM scores (γ)** to rank and select the best prediction. The parameters α & γ are carefully tuned to achieve the lowest WER, more details on this follow in the experiments and results section.

3. Experiments and Results

For modular analysis of the entire brain-to-text architecture, we isolate the problem into 2 discrete parts with their

separate metrics and analysis: neural signals to phoneme prediction followed by predicted phonemes to complete sentences.

3.1. Neural Encoding to Phonemes

We evaluate the efficacy of various sequential DL architectures for phoneme prediction from neural data. This intermediate phoneme mapping offers several advantages; provides a resource-efficient (41 target labels) and vocabulary-independent representation of the input. This is quantified using the Phoneme Error Rate (PER) derived from the Connectionist Temporal Classification (CTC) loss [13]. The CTC objective aggregates the predicted temporal sequence via path marginalization, followed by the removal of repeated labels and the designated blank symbol. However, Phoneme boundaries are often acoustically and perceptually ambiguous, lacking discrete segmentation in continuous speech. Also, similar PER values can correspond to widely divergent Word Error Rate (WER), indicating this metric is not without flaws.

To further alleviate the risks above, we analyze the confidence probabilities and confusion matrices across phoneme and phoneme categories, to infer strengths/ weaknesses of the decoder model.

3.1.1 Model Architecture Experiments

We begin by comparing six different recurrent architectures (unidirectional and bidirectional vanilla RNN, GRU, and LSTM) each trained for 24,000 batches. The resulting validation CTC loss and PER curves can be referred to in Appendix (Fig. 4). Among the six RNN architectures trained, the Bi-LSTM achieved the lowest validation PER (11.1%), outperforming the Bi-GRU (11.4%) and vanilla RNNs (13.3%).

However, we also note that the unidirectional GRU with lowest validation CTC loss is also a robust architecture, which we plan to investigate further in future work.

3.1.2 Data Augmentation Experiments

To address the issue of noisy and nonstationary neural signals, we introduce several data augmentation strategies, such as random time mask and random channel dropout, which not only reduce the session variance, but also reduce the overall PER from 9.99% (Fig. 5) to 7.89% (Fig. 2) on the validation set. The PER validation & Loss curves for the architecture-benchmark (Fig. 6a, 6b) & the benchmark (post data augmentation - (Fig. 6c, Appendix Fig. 6d)) are shared in A.3.1 (We also share download links for both checkpoints in the same section).

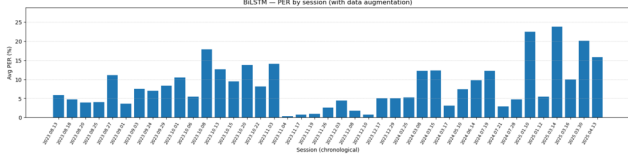


Figure 2: Bi-Directional LSTM, Session wise Variance on benchmark model (with data augmentation - 7.89% PER).

3.1.3 Ablation Study

To evaluate the contribution of different subsets of neural signals, we split the 512-dimensional feature vector into eight contiguous 64-dimensional blocks, each corresponding to a specific cortical area (ventral 6v, area 4, 55b, dorsal 6v) and signal type (threshold crossings and spike band power). Ablation experiments on these blocks (Fig. 3) show that removing spike band power consistently produces a larger increase in PER than removing thresholding crossings in all the four cortical areas. This pattern suggests that spike band power may carry more informative activities so the decoder relies on it more heavily for accurate phoneme prediction.

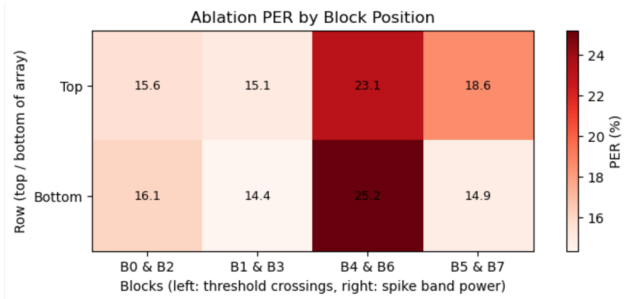


Figure 3: Ablation PER by Block Position

3.2. Phonemes to Text

This stage involves transformation of predicted phoneme sequences into complete, coherent linguistic units (sentences), with the metric being the Word Error Rate (WER).

While WER provides an interpretable measure, it must be looked relative to the associated PER to accurately diagnose the system’s performance bottleneck—whether the error originates in the initial neural-to-phoneme mapping or the subsequent phoneme-to-text language model.

It is crucial to note that for end-to-end brain-to-text architectures that bypass the phonemic intermediate step, the WER metric alone can be misleading. Specifically, it only quantifies direct lexical matches, failing to capture the semantic similarity or overall intelligibility between the predicted and target sentences.

3.2.1 Baseline + Lattice Re-scoring with LLMs

Across all decoding settings, lattice re-scoring with the LLM produced consistent reductions in WER relative to the corresponding n -gram baselines. The magnitude of improvement was largest for the 1-gram decoder, where the initial lattice contains many locally plausible but globally incoherent candidates; LLM scoring effectively suppresses these hypotheses and elevates sentences that exhibit better long-range structure. For the 5-gram decoder, LLM re-scoring improved the **WER** from **11.79%** to **10.76%**. This represents **our best overall performance**, demonstrating that *lattice rescoring provides additive value even when applied to strong n -gram baselines*. You can find the improvement measures over the baseline and sample sentence differences in [A.5.1](#)

Overall, these experiments show that LLM lattice re-scoring systematically strengthens both lexical accuracy and sentence-level coherence, with the greatest relative benefit observed when the underlying n -gram model is weak but with persistent improvements even when a strong baseline is used.

3.2.2 Decoder (GPT2) vs Encoder-Decoder (T5-Base) Experiments

All experiments were conducted on a compute node equipped with 4x NVIDIA A100 (40GB) GPUs. Given this high-memory environment, we performed full fine-tuning of the models without the need for quantization or LoRA adapters. This allowed us to maximize the representational capacity of the pre-trained weights for the phoneme-to-text translation task.

Hyper-Parameter Selection

Our hyperparameter configurations were selected based on established guidelines for transfer learning in Transformer-based architectures [19, 18, 5]. We standardized the training regimen across both architectures to ensure a fair comparison. We performed full fine-tuning on 4x NVIDIA A100 (40GB) GPUs for 3 epochs, a duration observed to maximize adaptation without overfitting. We standardized the training regimen across architectures using a learning rate of 5×10^{-5} with a linear decay scheduler. To stabilize gradient estimates given the noisy phonetic inputs, we utilized an effective global batch size of 128 (per-device batch size of 8 with 4 gradient accumulation steps). Finally, a context window of 1024 tokens was selected to accommodate long phonetic sequences without truncation.

Performance of Models on Validation Set

We trained GPT2 & T5-Base on both augmented and clean datasets. The cross entropy loss curves for both train and validation can be found in the Appendix: (Fig. 11, 12).

Upon coupling these models with our architecture-benchmark model from the previous stage (bi-directional LSTM), we observed the following word error rates on the validation set for all 4 approaches:

- T5-Base: T5 is a pre-trained "Language Expert." When data augmentation hampered the phonetic signal (e.g., deleting critical phonemes), T5 didn't try to output a "broken" sentence. Instead, it used its massive internal language model to hallucinate a grammatically perfect sentence that didn't correlate well with input.
 1. Dataset with Augmentation: 24.46% WER
 2. **Clean Dataset: 20.51% WER**
- GPT2: GPT-2 (Decoder-Only) treats the task as "Sequence Completion." It is less prone to rewriting the whole sentence based on global context and is more forced to attend to the local phoneme tokens defined in the input.
 1. **Dataset with Augmentation: 17.15% WER**
 2. Clean Dataset: 20.36% WER

Choosing the best performing version from each of the two models, we next coupled these with our PER-Benchmark Model from the previous stage (bi-directional LSTM + time-masking) and apply acoustic rescoring to the end-to-end pipeline in order to achieve the best WERs from these models. This is discussed in the next section.

3.3. Acoustic Rescoring Results

As a final step to our neural-encoding to text prediction pipeline, we combine our best models from both stages: *Stage 1 (bi-directional LSTM + time-masking)* + *Stage 2 (GPT2 with Data Augmentation & T5-Base with Clean Dataset)*, and apply acoustic re-scoring to squeeze out the best performance. Our hyperparameter search grid varies over the range of $[0.33, 1]$ for both α (weight given to log probability of phoneme sequence) & γ (weight given to log probability of predicted sentence) parameters, the results for the same can be found in Appendix (Table. 4). All experiments were done keeping the beam width limited to 100 & top k limited to 50. We chose `Mistral-7B-v0.1` [15] as the LLM to score sentence confidence (weighted by γ).

We achieved a benchmark of **13.35% WER** with GPT-2 ($\alpha : 1.0, \gamma : 0.6$) & 17.18% WER with T5-Base ($\alpha : 0.66, \gamma : 1.0$).

4. Conclusion

In this work, we successfully demonstrated that integrating Large Language Models via acoustic lattice rescoring significantly enhances the decoding of speech from neural activity. Our best-performing configuration—a 5-gram decoder coupled with LLM rescoring—achieved a Word Error Rate (WER) of 10.76%, improving upon the strong n-gram baseline of 11.79%. This result validates that lattice rescoring provides additive value even when applied to high-capacity language models, efficiently bridging the gap between acoustic fidelity and linguistic coherence.

A critical insight from our architectural comparison is the trade-off between generative capability and conditional adherence. While the encoder-decoder T5 model produced fluent text, it exhibited a tendency toward "semantic hallucination," frequently overriding noisy phonetic inputs with grammatically correct but factually unrelated sentences. In contrast, the decoder-only GPT-2 architecture maintained tighter alignment with the neural-to-phoneme output. This distinction suggests that for BCI applications where acoustic precision is paramount, "smaller" or more constrained language models may be preferable to massive generative models prone to hallucinations.

However, these results must be contextualized within significant computational and data-intrinsic challenges. From the outset, we faced the 300 GB RAM requirement of the high-capacity 5-gram LM, which limited our ability to deploy straightforward WFST-based rescoring at scale. Beyond compute limits, the neural data presented intrinsic challenges due to electrode drift and non-stationarity, where standard supervised training proved insufficient. To address this, we investigated self-supervised normalization methods like BYOL [11] and SimSiam [4] to learn day-invariant representations. Looking forward, we identify the Conformer architecture [12] as an ideal solution to replace fragile patching hyper-parameters, utilizing its hybrid convolution and self-attention modules to robustly capture global context.

5. Work Division

Work division among all 4 of us can be referred to in Table 1.

Student Name	Contributed Aspects	Details
Ozgur Mert Emir	Both Stages	Phonemes to Sentences conversion (Lattice Re-Scoring, Dataset Creation)
Qianyi Miao	Both Stages	Sectional Metrics/ Visualization/ Ablation Study/ Data Augmentation (Stage-1)
Rahul Kumar Agrawal	Both Stages	Sectional Metrics/ Visualization/ RNN Architectures
Rishabh Sood	Both Stages	Phonemes to Sentences conversion (T5/GPT2, Dataset Creation) / Data Augmentation (Stage-2)

Table 1: Contributions of team members. Both Stages refer to Neural Encoding $\rightarrow$ Phoneme $\rightarrow$ Text.

References

- [1] Nicholas Card, Maitreyee Wairagkar, Carrina Iacobacci, et al. Data for: An accurate and rapidly calibrating speech neuroprosthesis, 2025. [Dataset]. [1, 3](#)
- [2] Nicholas S. Card, Maitreyee Wairagkar, Carrina Iacobacci, Xianda Hou, Tyler Singer-Clark, Francis R. Willett, Erin M. Kunz, Chaofei Fan, Maryam Vahdati Nia, Darrel R. Deo, Aparna Srinivasan, Eun Young Choi, Matthew F. Glasser, Leigh R. Hochberg, Jaimie M. Henderson, Kiarash Shahlaie, Sergey D. Stavisky, and David M. Brandman. An accurate and rapidly calibrating speech neuroprosthesis. *New England Journal of Medicine*, 391(7):609–618, 2024. [1, 2, 3](#)
- [3] Nicholas S. Card, Maitreyee Wairagkar, Carrina Iacobacci, Xianda Hou, Tyler Singer-Clark, Francis R. Willett, Erin M. Kunz, Chaofei Fan, Maryam Vahdati Nia, Darrel R. Deo, Aparna Srinivasan, Eun Young Choi, Matthew F. Glasser, Leigh R. Hochberg, Jaimie M. Henderson, Kiarash Shahlaie, Sergey D. Stavisky*, and David M. Brandman*. nejm-brain-to-text. <https://github.com/Neuroprosthetics-Lab/nejm-brain-to-text>, 2024. [2, 3, 9, 10](#)
- [4] Xinlei Chen and Kaiming He. Exploring simple siamese representation learning, 2020. [6](#)
- [5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019. [5](#)
- [6] Susan Koch Fager, Melanie Fried-Oken, Tom Jakobs, and David R. Beukelman. New and emerging access technologies for adults with complex communication needs and severe motor impairments: State of the science. *Augmentative and Alternative Communication*, 35(1):13–25, 1 2019. [1](#)
- [7] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The Pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020. [2, 3](#)
- [8] Arnab Ghoshal, Gilles Boulianne, Lukáš Burget, Ondrej Glembek, Nagendra Goel, Mirko Hannemann, Petr Motlíček, Yanmin Qian, Petr Schwarz, Jan Silovsky, Georg Stemmer, and Karel Vesel. The kaldi speech recognition toolkit. *IEEE 2011 Workshop on Automatic Speech Recognition and Understanding*, 01 2011. [2, 3, 9](#)
- [9] John J. Godfrey, Edward C. Holliman, and Jane McDaniel. SWITCHBOARD: Telephone speech corpus for research and development. In *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, pages 517–520, San Francisco, CA, USA, 1992. Also released by the Linguistic Data Consortium as Switchboard-1 Release 2 (LDC97S62), DOI: 10.35111/sw3h-rw02. [2, 3](#)
- [10] Alex Graves, Santiago Fernández, Faustino Gomez, and Jürgen Schmidhuber. Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks. In *Proceedings of the 23rd International Conference on Machine Learning, ICML '06*, page 369–376, New York, NY, USA, 2006. Association for Computing Machinery. [2, 9](#)
- [11] Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre H. Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Daniel Guo, Mohammad Gheshlaghi Azar, Bilal Piot, Koray Kavukcuoglu, Rémi Munos, and Michal Valko. Bootstrap your own latent: A new approach to self-supervised learning, 2020. [6](#)
- [12] Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, and Ruoming Pang. Conformer: Convolution-augmented transformer for speech recognition, 2020. [6](#)
- [13] Awni Hannun. Sequence Modeling with CTC. *Distill*, 2(11), 11 2017. [4](#)
- [14] IEEE. IEEE Recommended Practice for Speech Quality Measurements. *IEEE Transactions on Audio and Electroacoustics*, 17:225–246, 1969. Appendix C contains the 1965 Revised List of Phonetically Balanced Sentences (Harvard Sentences). Also published as IEEE Standard 297-1969, DOI: 10.1109/IEEESTD.1969.7405210. [2, 3](#)
- [15] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Léo Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7b, 2023. [6](#)
- [16] Sean L. Metzger, Kaylo T. Littlejohn, Alexander B. Silva, David A. Moses, Margaret P. Seaton, Ran Wang, Maximilian E. Dougherty, Jessie R. Liu, Peter Wu, Michael A.

- Berger, Inga Zhuravleva, Adelyn Tu-Chan, Karunesh Ganguly, Gopala K. Anumanchipalli, and Edward F. Chang. A high-performance neuroprosthesis for speech decoding and avatar control. *Nature*, 620(7976):1037–1046, 8 2023. 1
- [17] Hoang Phuoc. Switchboard ASR Dataset. Hugging Face Dataset, 2022. Based on the Switchboard-1 Release 2 (LDC97S62) corpus. 3
- [18] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019. 3, 5
- [19] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020. 3, 5
- [20] Segyges. OpenWebText2. Hugging Face Dataset, 2021. 3
- [21] Rishabh Sood, Ozgur Mert Emir, Qianyi Miao, and Rahul Kumar Agrawal. Brain-to-text-25. <https://github.com/RishabhSood/brain-to-text-25>, 2025. 4
- [22] Francis Willett, Erin Kunz, Chaofei Fan, Donald Avansino, Guy Wilson, Eun Young Choi, Foram Kamdar, Matthew Glasser, Leigh Hochberg, Shaul Druckmann, Krishna Shenoy, and Jaimie Henderson. Data for: A high-performance speech neuroprosthesis, 6 2023. 1

A. Appendix

In this appendix, we report additional training, validation curves and other information for our neural-to-phoneme and phoneme-to-text models.

A.1. Baseline CTC + WFST n -gram Decoding: Formal Details

For completeness, we give the notation and scoring equations used in the baseline CTC–WFST decoding pipeline.

Let $x_{1:T}$ denote the sequence of neural features for a single trial, as produced by the RNN–GRU phoneme model’s preprocessing pipeline (day-specific linear layers and temporal patches).[3] The GRU emits framewise logits $z_t \in \mathbb{R}^V$ over a vocabulary of size V (phonemes plus the CTC blank) at each frame $t = 1, \dots, T$. We convert these logits to log posterior probabilities

$$\ell_t = \log \text{softmax}(z_t) \in \mathbb{R}^V,$$

optionally reordering the symbol indices to match the WFST graph’s labeling conventions (e.g., placement of the blank/SIL symbols).

Decoding is performed with a Kaldi-based C++ WFST decoder (`LatticeFasterDecoder`) over a composed TLG graph. Each arc e in the WFST carries a pair of log weights $(w_g(e), w_a(e))$ representing the graph/LM component and the acoustic component, respectively.[8] For a complete path π through the WFST, the total lattice cost is

$$C(\pi) = \sum_{e \in \pi} (w_g(e) + w_a(e)).$$

It is often convenient to separate these into

$$\text{ac}(\pi) = -\frac{1}{\lambda_a} \sum_{e \in \pi} w_a(e), \quad \text{lm}(\pi) = -\sum_{e \in \pi} w_g(e),$$

where $\lambda_a > 0$ is the acoustic scale. Here $\text{ac}(\pi)$ is proportional to the log acoustic likelihood under the CTC model, while $\text{lm}(\pi)$ is the log probability under the n -gram language model (plus lexicon).

The baseline WFST decoder chooses a single best hypothesis by log-linearly combining these scores:

$$\hat{\pi}_{\text{base}} = \arg \max_{\pi} [\alpha \text{ac}(\pi) + \beta \text{lm}(\pi)],$$

with fixed $\alpha, \beta > 0$ that are implicitly controlled by the acoustic scale and LM-related decoding parameters (e.g., `--acoustic-scale`, beam width, and lattice pruning thresholds). The best-path lattice output is then mapped to a word sequence $\mathbf{w} = (w_1, \dots, w_T)$ via standard WFST symbol mapping and CTC collapse (removal of blanks and repeated labels).

In addition, we may extract an N -best list $\{\pi^{(k)}\}_{k=1}^N$ from the lattice and map each path to its corresponding word sequence $\mathbf{w}^{(k)} = (w_1^{(k)}, \dots, w_{T_k}^{(k)})$. A second-stage sentence-level language model (LLM) assigns each candidate a score

$$\tilde{L}^{(k)} = \log P_{\text{LLM}}(\mathbf{w}^{(k)}),$$

computed in a batched fashion over the N -best list. In the simplest configuration, the final decoded sentence is given by

$$\hat{\mathbf{w}}_{\text{final}} = \arg \max_k \tilde{L}^{(k)},$$

so that the CTC acoustic model and n -gram LM define the search space (the lattice), while the LLM acts purely as a reranker over the top- N WFST hypotheses.

This formulation closely mirrors standard hybrid ASR systems with CTC or HMM acoustic models and WFST n -gram decoding, augmented with second-pass neural language model rescoring.[10, 8]

A.2. Baseline + Lattice Rescoring with LLMs: Formal Details

A.2.1 Lattice export

We modify the baseline CTC+WFST decoder so that, in addition to outputting the best path, it exposes the full decoding lattice after search.[3, 8] Let $\mathcal{L} = (S, E)$ denote this lattice, where S is the set of states and E is the set of directed arcs. Each arc $e = (s \rightarrow s') \in E$ has

- an input label $i(e)$,
- an output label $o(e)$ (word ID or epsilon),
- a weight pair $(w_g(e), w_a(e))$, corresponding to graph/LM and acoustic components, respectively.

We export $\mathcal{L}$ to Python in a simplified form (start state, arc list, final states), preserving all weights.

A.2.2 Lattice as a word graph

We interpret $\mathcal{L}$ as a directed acyclic graph over word IDs. Using the `words.txt` symbol table from the baseline repository, we map word IDs to surface forms, treating special symbols such as `<eps>`, `<blk>` / `<blank>`, and `<sil>` as epsilon (i.e., they do not emit words).[3]

A partial hypothesis in our lattice search is defined by

$$h = (s, G, A, \mathbf{w}, L),$$

where

- $s \in S$ is the current lattice state,
- G and A are accumulated graph and acoustic costs,
- $\mathbf{w} = (w_1, \dots, w_t)$ is the current word prefix,
- $L = \log P_{\text{LLM}}(w_1, \dots, w_t)$ is the LLM log probability of the prefix.

We define

$$\text{ac}(A) = -\frac{A}{\lambda_a}, \quad \text{lm}(G) = -G,$$

where $\lambda_a > 0$ is the acoustic scale (as in the baseline). The intermediate score for h is

$$\text{Score}(h) = \alpha \cdot \text{ac}(A) + \beta \cdot \text{lm}(G) + \gamma \cdot L,$$

with hyperparameters $\alpha, \beta, \gamma > 0$ controlling the relative influence of the acoustic model, n -gram LM, and LLM.

A.2.3 Beam search over the lattice

We perform beam search over $\mathcal{L}$ under the above scoring function.

Initialization. We start at the lattice’s initial state s_{start} with an empty prefix:

$$B_0 = \{(s_{\text{start}}, G = 0, A = 0, \mathbf{w} = (), L = 0)\}.$$

Expansion. Given a beam B_t at step t , for each hypothesis $h = (s, G, A, \mathbf{w}, L) \in B_t$ and each outgoing arc $e = (s \rightarrow s')$, we compute

$$G' = G + w_g(e), \quad A' = A + w_a(e).$$

If $o(e)$ is epsilon or a special symbol, we set $\mathbf{w}' = \mathbf{w}$ and $L' = L$. If $o(e)$ is a real word u , we form the new prefix $\mathbf{w}' = (\mathbf{w}, u)$ and update

$$L' = \log P_{\text{LLM}}(\mathbf{w}'),$$

using the LLM in a prefix-cached fashion (i.e., storing intermediate LLM hidden states or prefix probabilities to avoid recomputation).

Each expanded hypothesis $h' = (s', G', A', \mathbf{w}', L')$ receives a score

$$\text{Score}(h') = \alpha \cdot \text{ac}(A') + \beta \cdot \text{lm}(G') + \gamma \cdot L'.$$

Pruning. All expanded hypotheses at step t are collected into a candidate set, and we retain only the top K hypotheses by Score to form the next beam B_{t+1} . The beam size K is a hyperparameter.

Finalization. If a state s is final in the lattice, with final weights $(w_g^{\text{final}}, w_a^{\text{final}})$, we treat any hypothesis $h = (s, G, A, \mathbf{w}, L)$ as a completed path by adding these final costs to (G, A) :

$$G_{\text{final}} = G + w_g^{\text{final}}, \quad A_{\text{final}} = A + w_a^{\text{final}}.$$

We then compute the final score

$$\text{Score}_{\text{final}}(h) = \alpha \cdot \text{ac}(A_{\text{final}}) + \beta \cdot \text{lm}(G_{\text{final}}) + \gamma \cdot L.$$

We collect all completed hypotheses over all steps and keep the top N by $\text{Score}_{\text{final}}$, yielding an N -best list

$$\left\{ (\mathbf{w}^{(k)}, \text{ac}^{(k)}, \text{lm}^{(k)}, \text{llm}^{(k)}) \right\}_{k=1}^N,$$

where $\mathbf{w}^{(k)}$ is the word sequence associated with the k -th path, $\text{ac}^{(k)}$ and $\text{lm}^{(k)}$ are derived from $(A_{\text{final}}, G_{\text{final}})$, and $\text{llm}^{(k)} = \log P_{\text{LLM}}(\mathbf{w}^{(k)})$.

The top-1 hypothesis under this lattice+LLM search is therefore

$$\hat{\mathbf{w}}_{\text{lattice+LLM}} = \arg \max_{\mathbf{w}} [\alpha \text{ac}(\mathbf{w}) + \beta \text{lm}(\mathbf{w}) + \gamma \text{llm}(\mathbf{w})],$$

where the maximization is over all paths in the original lattice, not just the C++ decoder’s N -best list.

A.2.4 Relation to sentence-level LLM rescoring

For completeness, note that the same sentence-level LLM rescoring used in the baseline CTC+WFST system can be applied on top of the lattice+LLM N -best list simply by re-ranking $\{\mathbf{w}^{(k)}\}$ using a pure LLM score $\log P_{\text{LLM}}(\mathbf{w}^{(k)})$. In our experiments, we evaluate configurations with and without this final sentence-level step to isolate the contribution of the lattice-level integration itself.

A.3. Stage 1: Neural Encodings to Phonemes

A.3.1 Model Architecture Experiments

Plots for PER & CTC Loss Comparison across 6 RNN Architectures: Fig. 4.

A.3.2 Bi-Directional LSTM - Architecture Benchmark

Download Model Weights [here](#)

A.3.3 Bi-Directional LSTM - PER Benchmark

Download Model Weights [here](#)

A.4. Embeddings Visualization

The visualization of the bi-directional LSTM’s learned embeddings for each phoneme helps mitigate reliance solely on the PER metric as well as to evaluate the feasibility of directly incorporating these embeddings into Stage 2 (Phoneme-to-Text conversion), as an alternative to conventional labeled one-hot encoding.

We employed t-SNE to visualize hidden states, color-coded by their corresponding Phoneme IDs. The aggregated visualization across all sessions (Appendix: Fig. 7 left) presents no distinct, separable clustering pattern. This is despite the model achieving empirically strong performance metrics (good PERs), leading to the hypothesis that the issue lies in the information loss inherent in projecting the high-dimensional, dense representations onto a two-dimensional space.

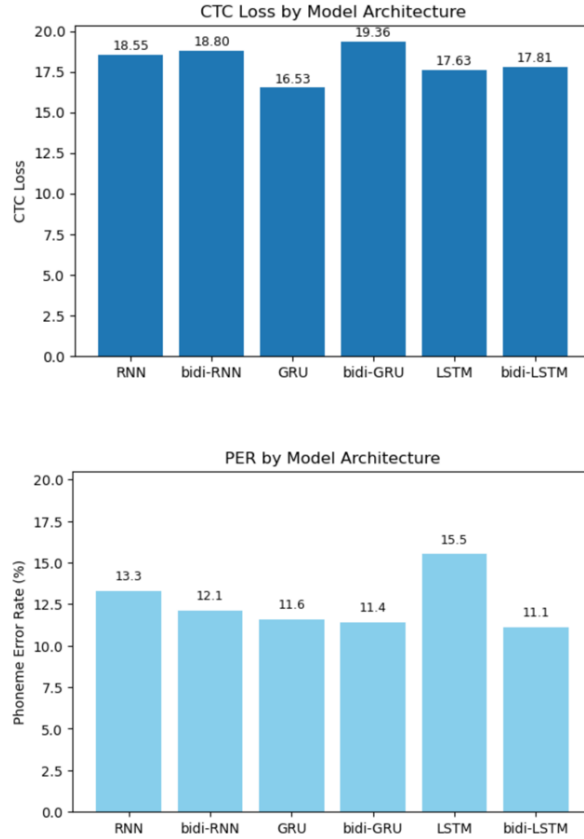


Figure 4: CTC Loss & PER Comparison across 6 RNN architectures

However, restricting the analysis to a single-session subset of the data reveals a significantly different structure (Appendix: Fig. 7 right). This session-specific plot exhibits more clustered, caterpillar-like representations. This contrast strongly indicates that the learned representations are widely divergent across recording sessions. This variability in the neural features further reinforces the need for an explicit normalization mechanism to account for session-specific data characteristics, a challenge that is discussed in Section 4, Future Work.

A.5. Stage 2: Phoneme to Text

A.5.1 Baseline + Lattice Re-scoring Results

Table 2: WER comparison between baseline decoding and decoding with LLM lattice re-scoring.

Model	Baseline WER (%)	+ Lattice Re-scoring WER (%)
1-gram decoder	49.67	24.51
5-gram decoder	11.79	10.76

A.5.2 Filtered Dataset

The final dataset files can be [found here](#).

Table 3: Sample outputs comparing ground truth, 1-gram baseline, and lattice re-scored predictions.

Ground Truth	1-gram Baseline	1-gram + Lattice Re-scoring	5-gram + Lattice Re-scoring
You can see the code at this point as well.	yu can sci the kodak this pointers	you can see the code at this point as	you can see the code at this point as well
You can come and go as you please.	yu can cum undergo has ewe	you can come and go as you	you can come and go as you
Not in terms of north and south.	knotted terms of north indus	not in terms of north and south	not in terms of north and south

A.5.3 Training Data Format

GPT2

Input:
<BOS> [PHONEME_SEQUENCE] <SEP> [TEXT] <EOS>
Labels:
[IGNORE_TOKEN] × (len(PHONEME_SEQUENCE) + 2) + [TEXT] + <EOS>

T5-Base

Input:
"transcribe phonemes to text: DH AH | ..."
Labels:
"transcribed text"

A.5.4 Model Checkpoints

Checkpoints for GPT2 & T5 trained on both augmented & raw data can be [found here](#).

A.5.5 Acoustic Rescoring, Alpha-Gamma Tuning

Hyperparameter Search grid for Alpha-Gamma Tuning with results is shared in Table. 4.

A.6. Predicted Samples on GPT2 & T5 Benchmarks post Acoustic Rescoring

We also share representative prediction examples from both T5 & GPT2 coupled with bi-directional LSTM + tuned acoustic scoring parameters. Table 5 highlights where these models succeed and fall behind.

Table 4: Hyperparameter tuning results for Baseline Bi-LSTM and TimeMask models. The best Word Error Rate (WER) for each architecture is highlighted in bold. (We consider 50 best beams for all cases)

PhTT Model	RNN Checkpoint	Alpha (α)	Gamma (γ)	WER (%)
GPT2 + Data Aug	baseline_lstm_bi	1.0	0.33	15.50
		1.0	0.50	15.14
		1.0	1.00	14.61
		0.5	1.00	14.79
		0.33	1.00	15.09
		1.0	0.60	14.88
		1.0	0.75	14.71
		1.0	0.80	14.64
		0.6	1.00	14.63
		0.75	1.00	14.65
		0.8	1.00	14.69
	baseline_lstm_bi_timemask	1.0	0.33	13.75
		1.0	0.50	13.39
		1.0	1.00	13.50
		0.5	1.00	13.84
		0.33	1.00	14.03
		1.0	0.60	13.35
		1.0	0.75	13.39
		1.0	0.80	13.44
		0.6	1.00	13.82
		0.75	1.00	13.72
		0.8	1.00	13.64
T5-Base	baseline_lstm_bi	1.0	0.50	19.51
		1.0	1.00	19.09
		0.33	1.00	19.32
		0.66	1.00	19.15
		0.75	1.00	19.05
	baseline_lstm_bi_timemask	1.0	0.50	17.50
		1.0	1.00	17.28
		0.33	1.00	17.63
		0.66	1.00	17.18
		0.75	1.00	17.22

Table 5: Representative Prediction Examples from Both Models

Model	Predicted Phoneme Seq.	Prediction	Actual Text	α	γ
T5-Base (Good Examples)					
T5-Base	HH, AW, — , T, UW, — , AH, V, OY, D, ...	how to avoid being tracked on the internet	How To Avoid Being Tracked on The Internet	-1.76	-29.00
T5-Base	IH, T, — , IH, Z, — , AE, K, S, T, ER,...	it is estimated a total of three thousand properties were affected	It is estimated a total of three thousand properties were affected.	-4.74	-49.17
T5-Base	P, R, ER, F, EH, S, IH, N, Z, — , F, R,...	professors from other universities will present their research	Professors from other universities will present their research.	-5.42	-40.10
T5-Base	IH, T, — , D, AH, Z, — , N, AA, T, — ...	it does not snow as much as i remember while growing up	It does not snow as much as I remember while growing up.	-3.77	-46.16
T5-Base	B, IH, K, AO, Z, — , AH, — , L, AA, T,...	because a lot of our friends will be coming over later	Because a lot of our friends will be coming over later.	-0.01	-43.82
T5-Base (Poor Examples)					
T5-Base	P, L, OY, M, AH, N, T, — , N, K, AH, M,...	player name	Employment income.	-5.60	-20.59
T5-Base	IH, — , IY, Z, K, IH, — , DH, IH, Z, ...	I eat this it is he	I hope everything works out up there.	-5.61	-39.77
T5-Base	AH, — , V, EH, JH, T, AH, B, AH, L, — ...	a potentially dangerous	A vegetable garden.	-7.04	-24.96
T5-Base	F, AH, N, IY, — , P, AE, S, T, — , D, ...	funipaspaspa....	funny past decisions dallas future bring.	-7.42	-57.33
T5-Base	Y, UW, AY, IH, T, IY, — , D, IH, P, AA,...	uiiiiiiiiiiii	Zoology department.	-5.40	-23.33
GPT-2 (Good Examples)					
GPT-2	DH, EH, S, Z, — , AY, — , AE, M, — , ...	Yes, I am talking to you.	Yes, I am talking to you.	-3.80	-20.21
GPT-2	HH, AW, — , T, UW, — , AH, V, OY, D, ...	how to avoid being tracked on the internet	How To Avoid Being Tracked on The Internet	-1.76	-29.00
GPT-2	P, L, EY, IH, NG, — , T, R, AH, D, IH, ...	playing traditional used flat	playing traditional used flat	-0.92	-46.60
GPT-2	T, ER, M, Z, — , N, UW, Z, — , ER, N, ...	terms news earning by commitment	terms news earning by commitment	-1.36	-57.59
GPT-2	AA, R, — , IY, DH, ER, — , AO, R, — , ...	are either or both dogs considered dangerous under davis county ordinance	Are either or both dogs considered dangerous under Davis County ordinance?	-6.06	-66.29
GPT-2 (Poor Examples)					
GPT-2	R, AH, N, — , B, AA, T, AH, N, Z, — , ...	run boston olympics please please	run bargains hopeless persists meals	-5.12	-49.72
GPT-2	G, AA, D, AH, N, — , B, AY, — , V, OY,...	god forbid	Guided by voices.	-5.87	-21.71
GPT-2	— , IH, Z, — , S, AO, R, AH, B, EH, D,...	'is somebody	That is horrible.	-3.10	-27.67
GPT-2	Y, UW, AY, L, IH, T, IY, — , D, IH, P, ...	youll be disappointed	Zoology department.	-2.64	-26.23
GPT-2	IH, — , IY, Z, K, IH, — , IH, Z, — , ...	it is it is it is	I hope everything works out up there.	-4.93	-24.41

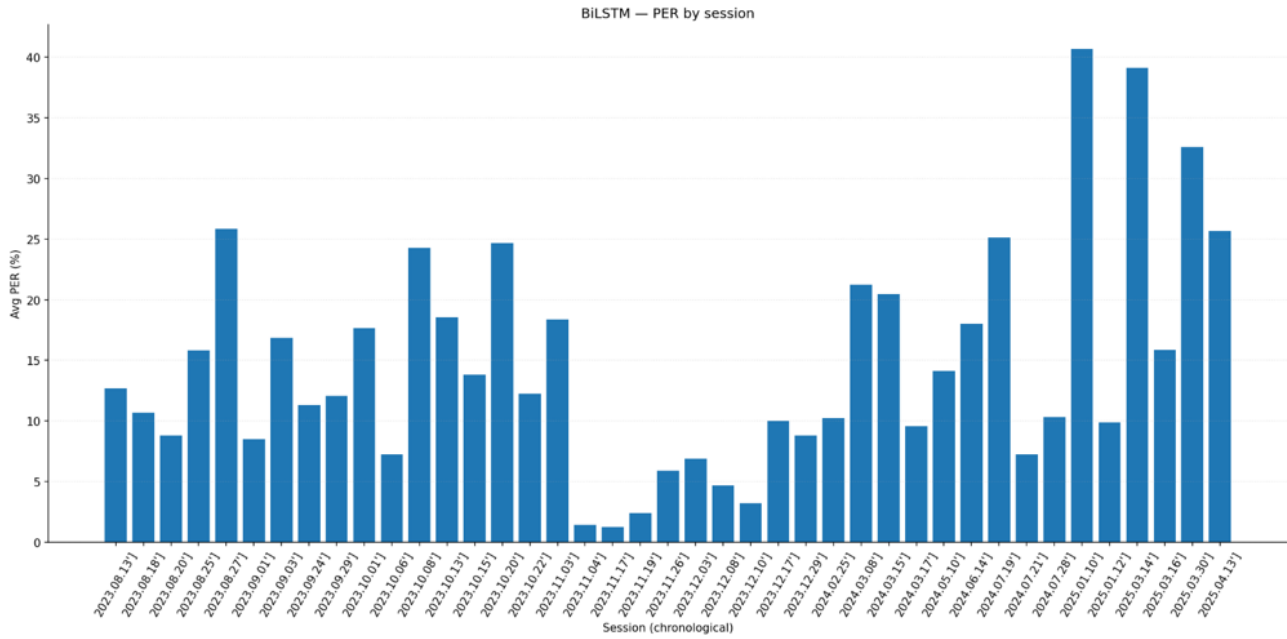
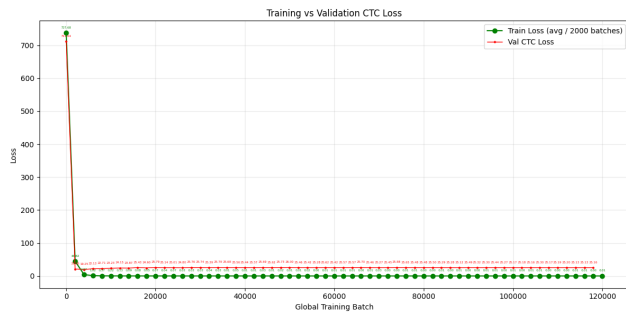
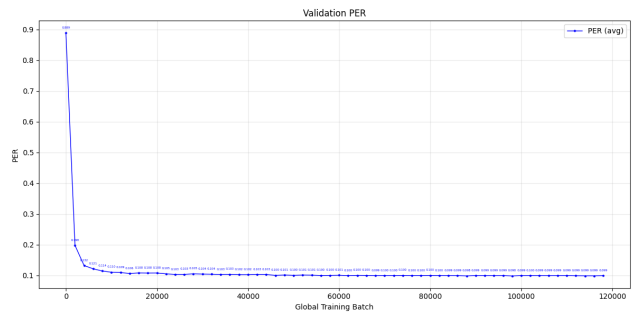


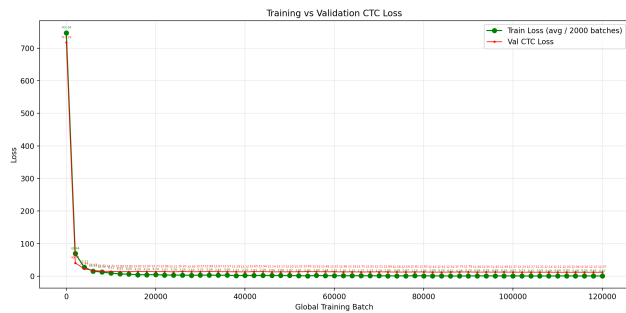
Figure 5: Bi-Directional LSTM, Session wise Variance on benchmark model (without Data Augmentation - 9.99% PER).



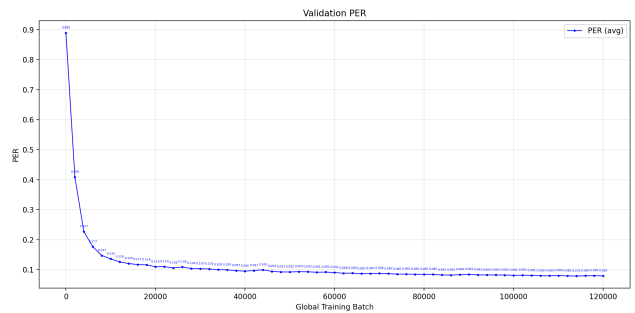
(a) Train-Val CTC Loss (120K epochs)



(b) Validation PER (120K epochs)



(c) CTC Loss with Data Augmentation



(d) Validation PER with Data Augmentation

Figure 6: Performance Comparison: Bi-Directional LSTM Benchmark Model (9.99% PER) vs. Data Augmentation Model (7.89% PER). Top row shows standard training; bottom row shows training with augmentation.

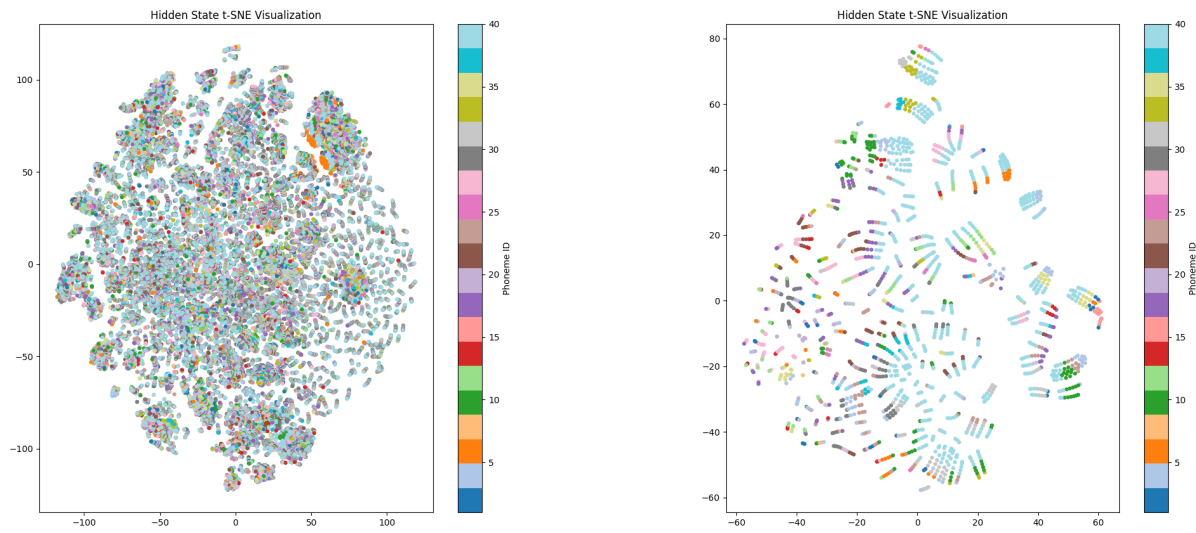


Figure 7: Hidden State t-SNE Visualization. The general plot (Left) shows no clear pattern, while a single session plot (Right) reveals more clustered representations.

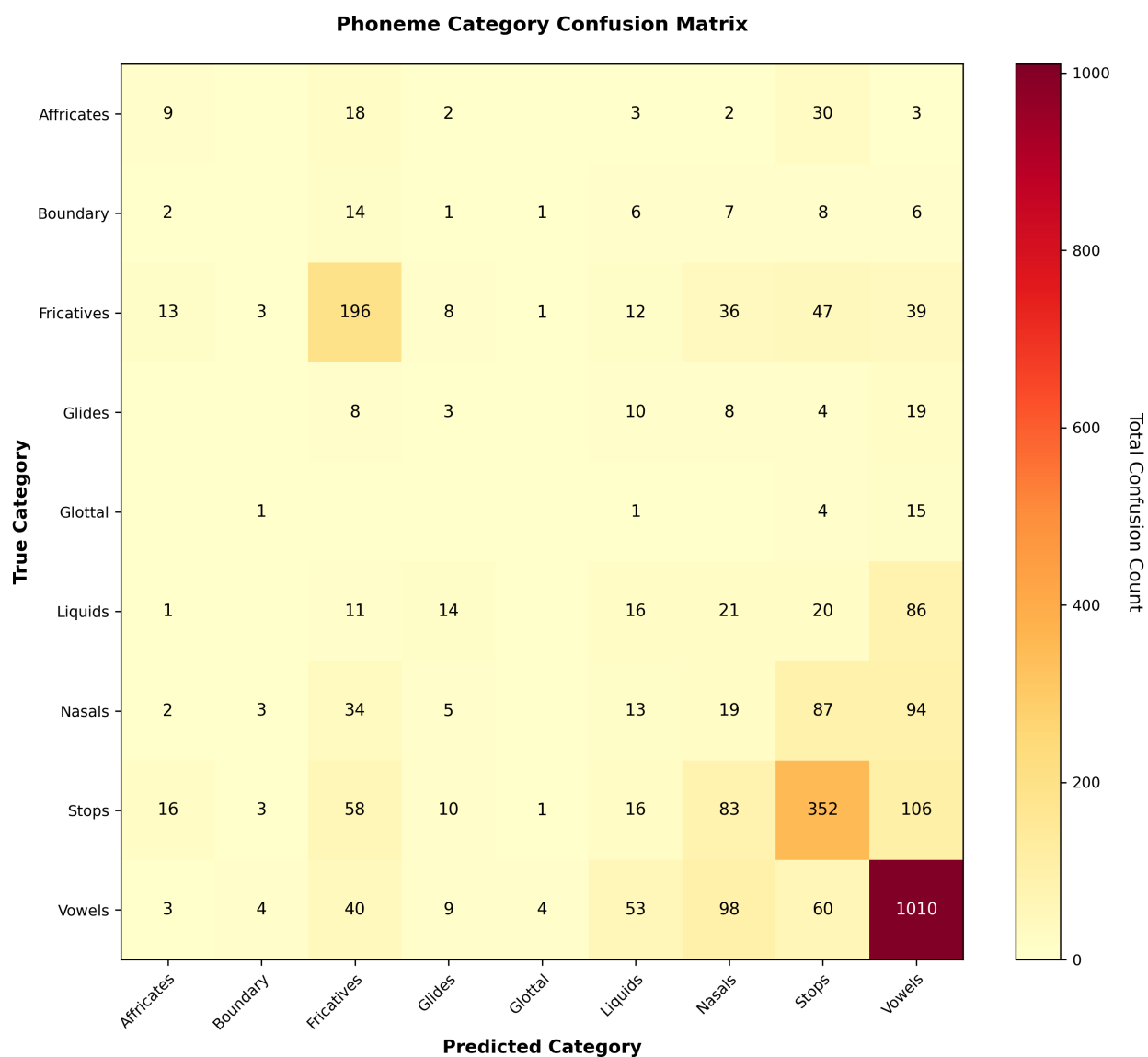


Figure 8: Phoneme Category confusion Matrix, Analyzed on top of validation set phoneme-prediction by our best performing baseline: Bi-Directional LSTM RNN Layer

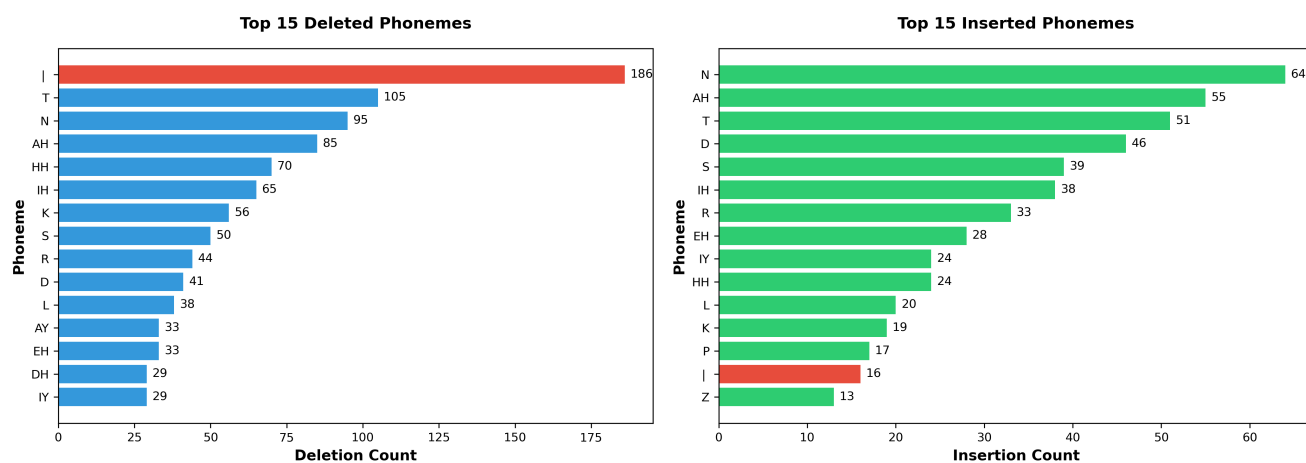


Figure 9: Phoneme Insertion-Deletion Frequency, Analyzed on top of validation set phoneme-prediction by our best performing baseline: Bi-Directional LSTM RNN Layer

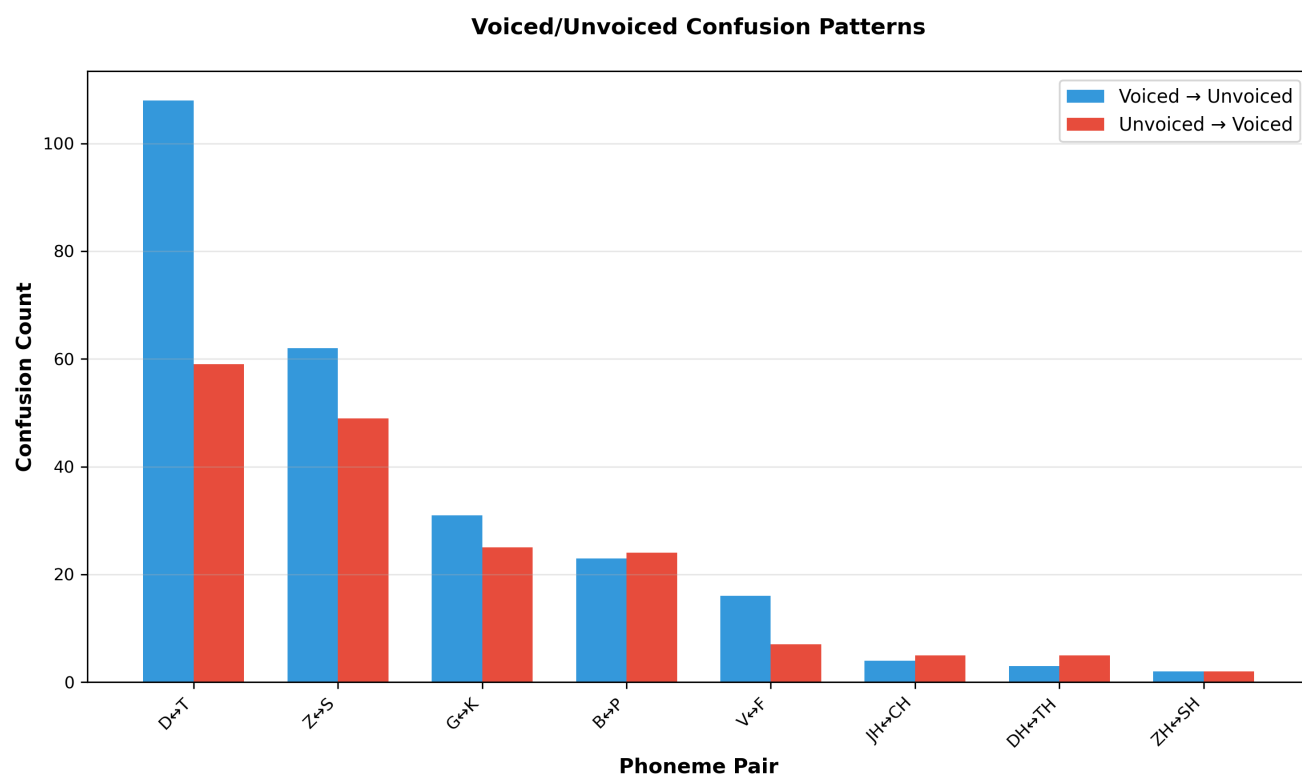


Figure 10: Voiced-Unvoiced Confusion, Analyzed on top of validation set phoneme-prediction by our best performing baseline: Bi-Directional LSTM RNN Layer

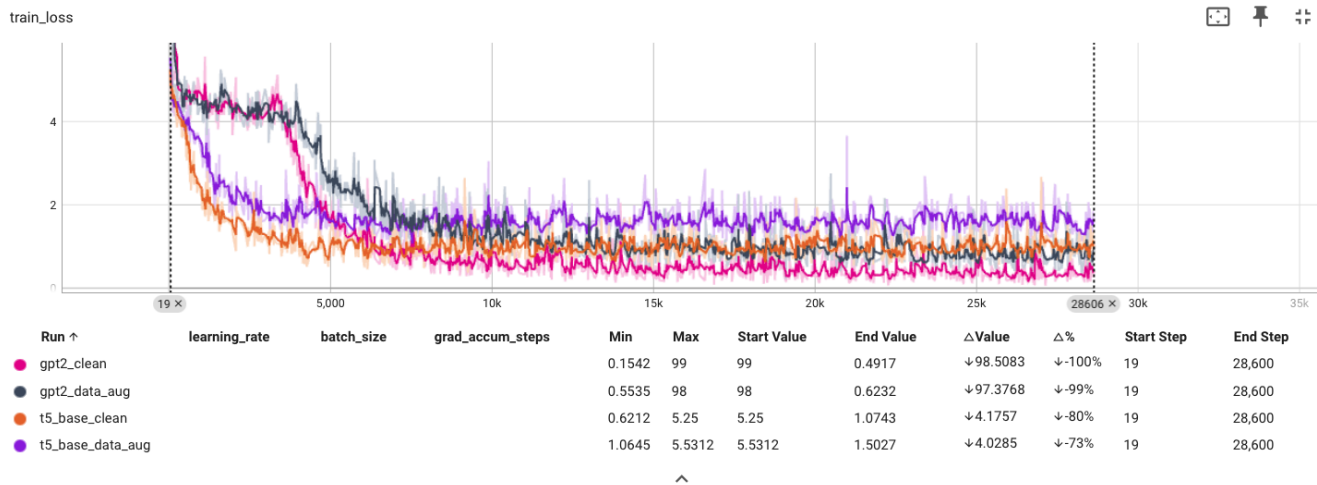


Figure 11: Cross Entropy Loss Curves (Train) for GPT-2 & T5 Models (with augmented / clean data) trained using *accelerate* on 4 NVIDIA A-100 GPUs

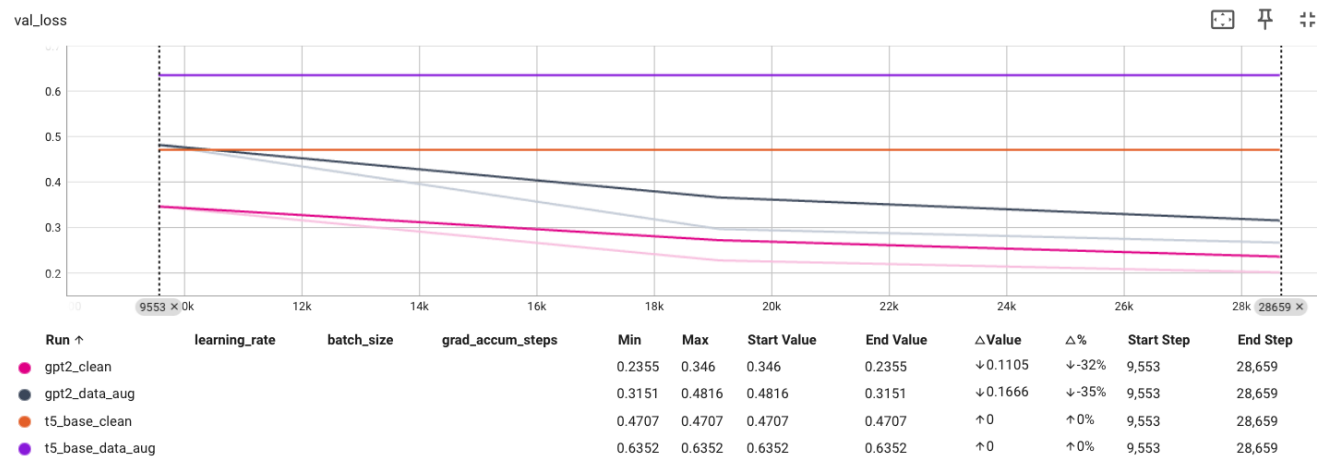


Figure 12: Cross Entropy Loss Curves (Validation) for GPT-2 & T5 Models (with augmented / clean data) trained using *accelerate* on 4 NVIDIA A-100 GPUs